



Lecture 19: Dynamic Inline Styles

INTERVIEW QUESTION

[> CORE](#)

How do you set inline styles dynamically, including CSS custom properties?

You need an element's color to come from a variable, or a CSS variable like `--columns` to be set from JavaScript. How do you drive inline styles from state?

While Svelte's `class:` directive is the best tool for toggling predefined styles, there are times when styles must be highly granular or completely unpredictable, such as setting a height to a user's exact mouse coordinate, or applying a custom brand color pulled from an API.

The Problem with Style Strings

Imagine you are building a dashboard where the user can pick a custom hex color for their profile banner.

If you try to achieve this using raw JavaScript DOM manipulation or raw HTML string concatenation, it quickly becomes error prone. In older frameworks, you might have to assemble strings manually like `style="background-color: " + userColor + "; opacity: " + currentOpacity`. This is visually messy, easy to break with a missing semicolon, and highly susceptible to malicious code injection if the user inputs something unexpected instead of a color.

The `style:` Directive

Svelte provides a `style:` directive that allows you to safely and declaratively bind reactive JavaScript variables directly to specific CSS properties.

The syntax is `style:property={value}` .



```
1  <script>
2    let themeColor = $state("#ff0000");
3    let opacity = $state(0.8);
4  </script>
5
6  <div
7    class="banner"
8    style:background-color={themeColor}
9    style:opacity={opacity}
10  >
11    User Profile
12  </div>
```

Binds background to the hex code

When `themeColor` or `opacity` change, Svelte surgically updates only those specific CSS properties on the element's inline style object, leaving any other styles completely untouched.

CSS Custom Properties (Variables)

The true superpower of the `style:` directive is how seamlessly it integrates with CSS Custom Properties (often called CSS variables).

Imagine you are building a dynamic grid layout where the user can choose how many columns they want via a slider. Instead of trying to write inline styles for every single grid child, you can use the `style:` directive on the parent container to define a CSS variable.

```
Grid.svelte
1  <script>
2    let columns = $state(3);
3  </script>
4
5  <div class="grid" style:--col-count={columns}>
    Sets the CSS variable natively on the parent container
6    <div class="item">1</div>
7    <div class="item">2</div>
8    <div class="item">3</div>
9  </div>
10
11  <style>
12    .grid {
13      display: grid;
14      grid-template-columns: repeat(var(--col-count), 1fr); /* ✓ The CSS re
15    }
16  </style>
```

This pattern is incredibly powerful. It allows your JavaScript to safely pass data over the fence to your CSS, letting the CSS engine do what it does best (layout and rendering) without polluting your markup with massive inline style strings.

Summary

When styles cannot be handled by static CSS classes because they rely on exact numerical values or dynamic user data, use Svelte's `style:` directive. It allows you to bind reactive state directly to CSS properties, or, more powerfully, to CSS Custom Properties (variables). This bridges the gap between your JavaScript state and your stylesheets safely and declaratively.

	RAW STRINGS (Legacy)	STYLE DIRECTIVE (Modern)
Syntax	<code>style="color: {val}"</code>	<code>style:color={val}</code>
Readability	Messy and error prone	Clean and declarative
CSS Variables	Difficult to concatenate	Natively supported via <code>style:--var</code>